

Module 1 (Foundations) — Post-Lecture Coding Questions

Instructions

- Use Python 3.10+. You may use `random`, `math`, and `statistics`.
- Keep your code readable (functions, docstrings, comments).
- For questions involving randomness, run multiple episodes and report summary statistics.
- You may submit a single `.py` file or a `.ipynb` notebook.

Shared Toy MDP (used in all questions)

We use an episodic, deterministic Markov Decision Process with:

- **States:** {A, B, T} where T is terminal.
- **Actions:** {0, 1}.
- **Discount factor:** $\gamma = 0.9$.

Dynamics and Rewards

State	Action	Next State	Reward
A	0	B	+1
A	1	T	0
B	0	A	0
B	1	T	+2

Stochastic Policy π (used in Q1–Q2)

- At A: $\pi(0|A) = 0.7$, $\pi(1|A) = 0.3$.
- At B: $\pi(0|B) = 0.4$, $\pi(1|B) = 0.6$.

Discounted Return

For an episode that generates rewards $r_1, r_2, \dots, r_T$, the discounted return from time $t = 0$ is:

$$G_0 = \sum_{k=0}^{T-1} \gamma^k r_{k+1}$$

Question 1 — Simulate Episodes and Compute Returns (Monte Carlo)

Goal: Practice generating trajectory data and computing discounted returns.

Tasks

1. Implement the environment step function: $(s_{t+1}, r_{t+1}, \text{done}) = \text{step}(s_t, a_t)$.
2. Implement the given stochastic policy $\pi(a|s)$ and sample actions from it.
3. Simulate N episodes starting from $s_0 = A$.
4. Compute G_0 for each episode and report:
 - the mean return,
 - the standard deviation of returns.

Hints

- Store each transition as a tuple $(s, a, r, s', \text{done})$ (useful later).
- The reward is received after taking an action.
- Discounted return in Python:

$$G_0 = \sum_{t=0}^{T-1} \gamma^t r_{t+1}$$

```
In [ ]: import random
from statistics import mean, pstdev

# --- Constants ---
GAMMA = 0.9          # Discount factor
N_EPISODES = 10_000  # Number of episodes to simulate
START_STATE = "A"    # Initial state

# Transition dynamics: (state, action) -> (next_state, reward, done)
TRANSITIONS = {
    ("A", 0): ("B", 1.0, False),
    ("A", 1): ("T", 0.0, True),
    ("B", 0): ("A", 0.0, False),
    ("B", 1): ("T", 2.0, True),
}

# --- Task 1: Environment step function ---
def step(state, action):
    # Look up the transition for the current (state, action) pair.
    # If not found (e.g. terminal state), default to staying terminal with 0 reward.
    return TRANSITIONS.get((state, action), ("T", 0.0, True))

# Example:
print("step('A', 0) ->", step("A", 0)) # Expected: ('B', 1.0, False)
```

```
print("step('B', 1) ->", step("B", 1)) # Expected: ('T', 2.0, True)
print("step('A', 1) ->", step("A", 1)) # Expected: ('T', 0.0, True)
```

```
step('A', 0) -> ('B', 1.0, False)
step('B', 1) -> ('T', 2.0, True)
step('A', 1) -> ('T', 0.0, True)
```

```
In [ ]: # Stochastic policy: state -> {action: probability}
POLICY = {
    "A": {0: 0.7, 1: 0.3},
    "B": {0: 0.4, 1: 0.6},
}

# --- Task 2: Stochastic policy and action sampling ---
def policy_probs(state):
    # Returns an empty dict for terminal states that have no defined policy.
    return POLICY.get(state, {})

# Example:
print("policy_probs('A') ->", policy_probs("A")) # Expected: {0: 0.7, 1: 0.3}
print("policy_probs('B') ->", policy_probs("B")) # Expected: {0: 0.4, 1: 0.6}
print("policy_probs('T') ->", policy_probs("T")) # Expected: {}

def sample_action(state):
    probs = policy_probs(state)
    actions = list(probs.keys())
    weights = list(probs.values())
    # Randomly pick one action weighted by its probability.
    return random.choices(actions, weights=weights)[0]

# Example: sample 10 actions from state A (should be mostly 0s due to 0.7 prob)
samples = [sample_action("A") for _ in range(10)]
print("sample_action('A') x10 ->", samples)

policy_probs('A') -> {0: 0.7, 1: 0.3}
policy_probs('B') -> {0: 0.4, 1: 0.6}
policy_probs('T') -> {}
sample_action('A') x10 -> [0, 1, 1, 0, 0, 0, 0, 0, 0, 1]
```

```
In [ ]: # --- Task 3: Simulate N episodes ---
def simulate_episode(start_state=START_STATE):
    state = start_state
    trajectory = [] # List of (s, a, r, s', done) tuples
    t = 0 # Time step counter used for discounting
    G0 = 0.0 # Accumulated discounted return

    while True:
        action = sample_action(state)
        next_state, reward, done = step(state, action)
        trajectory.append((state, action, reward, next_state, done))
        G0 += (GAMMA ** t) * reward
        t += 1

        if done: # Stop when terminal state is reached
            break
        state = next_state

    return trajectory, G0
```

```

# Example: print one episode's trajectory and return
example_trajectory, example_G0 = simulate_episode()
print("Example episode (s, a, r, s', done):")
for transition in example_trajectory:
    print(" ", transition)
print(f"Discounted return G0: {example_G0:.4f}")

# Run N episodes and collect returns
returns = []
for episode in range(N_EPISODES):
    _trajectory, G0 = simulate_episode()
    returns.append(G0)

```

```

Example episode (s, a, r, s', done):
  ('A', 0, 1.0, 'B', False)
  ('B', 1, 2.0, 'T', True)
Discounted return G0: 2.8000

```

```

In [9]: # --- Task 4: Compute and report G0 statistics ---
mean_return = mean(returns)
std_return = pstdev(returns)

print(f"Episodes simulated: {N_EPISODES}")
print(f"Mean return:          {mean_return:.4f}")
print(f"Std deviation:         {std_return:.4f}")

```

```

Episodes simulated: 10000
Mean return:        1.8939
Std deviation:      1.3692

```